

Contents

Подробный разбор архитектуры	2
1. Модель чтения архитектуры	2
2. Runtime object graph	2
3. ShowcaseApp: constructor и composition	3
4. Порядок инициализации ShowcaseApp	4
5. Состояние уровня приложения	4
6. Application Event Map	5
7. _stream() и _pipe()	5
8. Return protocol _pipe()	6
9. _pressStep() и keyboard modes приложения	7
10. Repeat filtering	7
11. Reverse input routing	8
12. Keyboard commands	8
13. Numeric / Shift audio track selection	9
14. Auxiliary mouse handling	9
15. Временное состояние правой кнопки	10
16. Визуальное состояние keyboard press	10
17. BaseSlider - граница ответственности	11
18. Состояние базового Slider	11
19. Lifecycle движения Slider	12
20. Почему это различие важно	12
21. PaginationSlider	13
22. Pagination и index synchronization	13
23. KeyboardSlider	14
24. Keyboard configuration как Strategy-like механизм	14
25. InfiniteSlider	15
26. Infinite loop teleport	15
27. Index normalization	15
28. DraggableSlider lifecycle	16
29. Drag trigger threshold	16
30. AutoscrollerSlider	17
31. Autoscroller State Machine	17
32. Почему LOCKED не равен OFF	17
33. Autoscroller resume gate	18
34. Autoscroller wake-up timing	18
35. Automatic vs manual movement	19
36. Autoscroller + dragging	19
37. AudioPlayer - граница домена	20
38. Public command surface AudioPlayer	20
39. AudioPlayer FSM	21
40. Почему native Audio state недостаточно	21
41. Album и track indices	22

42. Audio event model	22
43. Slider <-> Audio synchronization	23
44. Почему slidechange является synchronization event	23
45. Audio -> Slider synchronization	23
46. Album passthrough event	24
47. Album end	24
48. Input modes в ShowcaseApp	25
49. Special repeat behavior	26
50. Reverse pipeline	26
51. _pipe() как контракт	26
52. Почему это похоже на Chain of Responsibility	27
53. Inheritance + parent handler cooperation	27
54. Template Method-like structure	28
55. Static class contracts	28
56. Module constants vs static properties	28
57. this.constructor и polymorphic static lookup	29
58. Read-only static contracts	29
59. Shop и default URL	29
60. ButtonManager и KeyboardManager return contract	30
61. Mouse и Button pipeline	30
62. Auxiliary mouse input	30
63. Focus и keyboard UI state	31
64. Autoscroll и UI mode	31
65. Audio mode и UI mode	31
66. AudioDeckView	32
67. Audio track change	32
68. Audio timeline	32
69. Media Session	33
70. DOM validation как explicit contract	33
71. Почему fallback selector guessing избегается	34
72. Resize lifecycle	34
73. Dragging и Autoscroll interaction	34
74. Audio и Autoscroll - отдельные FSM	35
75. Configuration structure	35
76. Configuration как contract	35
77. Micro-framework characteristics	36
78. Что намеренно не абстрагировано	36
79. Почему State pattern не навязан	37
80. Более точная карта паттернов	37
81. Static inheritance в prototype implementation	38
82. Discipline для aliases	38
83. Prototype version и Class version	39
84. Маленькая внутренняя экосистема	39
85. Typical user interaction - Keyboard	40

86. Typical user interaction - Drag	40
87. Typical user interaction - Album playback	41
88. Main theme / Autoscroll interaction	42
89. Data flow	42
90. Component contracts	43
91. Error boundaries и validation	43
92. Почему архитектура сложнее обычного Slider	44
93. Architectural trade-off	44
94. Что делает архитектуру coherent	44
95. Архитектурное ядро	45
96. Final summary	46
97. Cross-Cutting Architectural Story	47
97.1 Почему все части действительно складываются вместе	47
97.2 Почему managers - больше, чем удобные wrappers	47
97.3 Почему _pipe() достоин отдельного описания	48
97.4 Почему локальные FSM здесь предпочтительнее	48
97.5 Почему архитектура не гонится за паттернами	49
97.6 Учебная цель	49
97.7 Поэтому проект работает как study object: UI даёт архитектуре конкретную задачу, а архитектура превращает UI в лабораторию, где OOP и system-design ideas можно исследовать без фреймворка, который скрывает большую часть механизмов.	49
98. Timer: зачем существует bootstrap	49
99. Как реально работает Timer.bootstrap	50
100. Почему bootstrap находится в Timer, а не в AutoscrollSlider	51
101. bootstrap и адаптивный autoscroll	51
102. albumend -> Slider -> slidechange -> AudioPlayer	52
103. Почему этот feedback loop не является проблемой	53
104. albumend является cancelable event	54
105. Active document и background document	55
106. Полный synchronization loop	55
107. Почему цикл не становится бесконечной цепочкой	56
108. Intent и Event не смешиваются	56
109. Pipeline и custom events - разные механизмы	57
109.1 Pipeline	57
109.2 Custom event	58
110. Одна интеракция может пройти через обе модели	58
111. Аналогичный round trip внутри AudioPlayer	59
112. Timer и event architecture встречаются в autoscroll	60
113. Media Session как ещё одна input boundary	60
114. Почему preventDefault() - не вся input architecture	61
115. State ownership в feedback loop	61
116. Система event-driven, но не “глобально event-based”	62
117. Ownership различных lifecycle	62
118. Почему маленькие объекты всё-таки образуют единую архитектуру	63
119. Эволюция архитектуры через реальные проблемы	64

120. Prototype/Class parity как отдельный архитектурный эксперимент . . . 64

121. Полная end-to-end картина 65

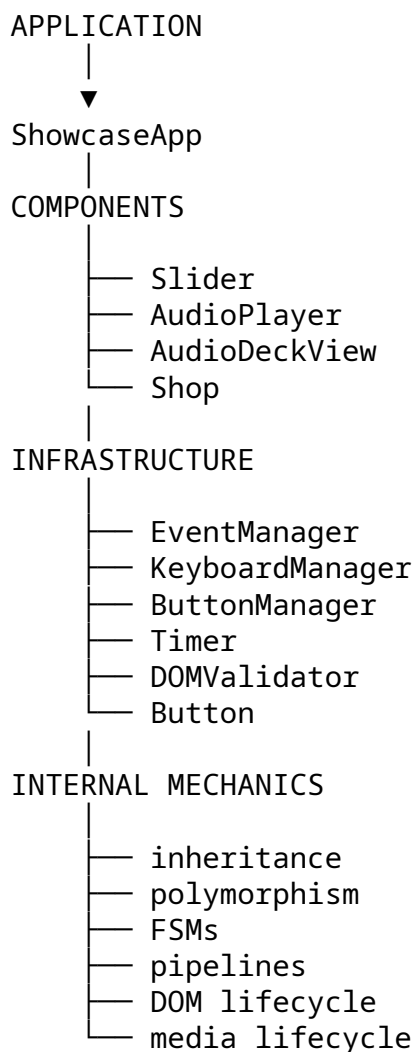
122. Заключительная перспектива Deep Dive 66

Подробный разбор архитектуры

Это implementation-level companion к Architecture-Overview.md. Его задача - объяснить не только, что делает каждая часть, но и зачем она существует, чем владеет, какой контракт предоставляет, как взаимодействует с соседями и как участвует в общих runtime-flow.

1. Модель чтения архитектуры

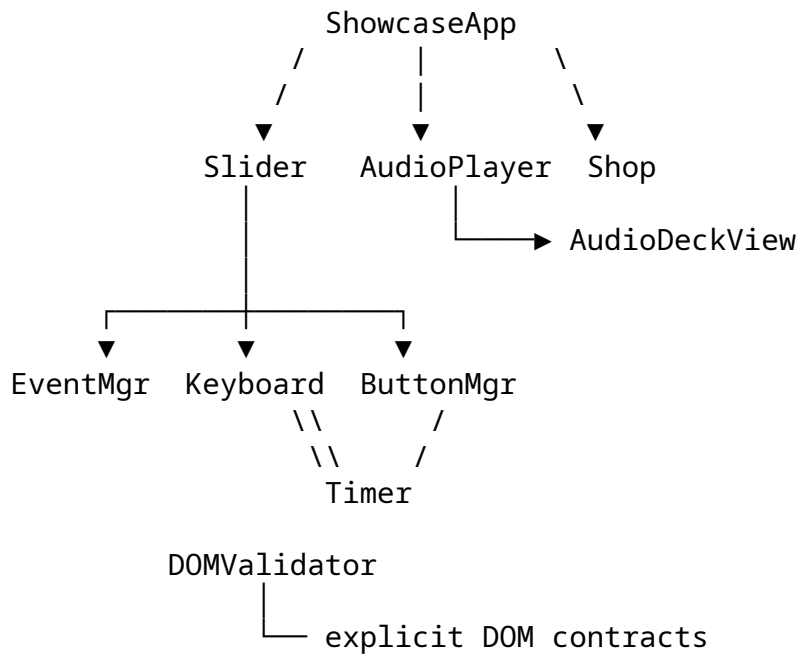
Систему проще всего понимать снаружи внутрь:



Главный вопрос на всём протяжении документа остаётся одним: **почему существует эта часть, чем она владеет и как она сотрудничает с соседними частями, не забирая их ответственность?**

3. Runtime object graph

Упрощённый runtime graph выглядит так:



Точная ownership-модель менеджеров следует границам компонентов и приложения, а не глобальному service locator.

4. ShowcaseApp: constructor и composition

Приложение получает основные компоненты:

```

slider
audioPlayer
audioDeckView
shop
options
  
```

и создаёт инфраструктуру уровня приложения:

```

_domValidator
_keyboardManager
_eventManager
  
```

Общая идея:

```

construction
  |
  v
composition root
  |
  ├── получить / передать компоненты
  ├── создать инфраструктуру
  └── выполнить initialization
  
```

Это практическое место, где встречаются IoC и composition.

5. Порядок инициализации ShowcaseApp

Порядок инициализации имеет значение.

```
ShowcaseApp.init()
├── _initDOMElements()
├── _initProps()
├── slider.init()
├── audioPlayer.init()
├── audioDeckView.init()
├── shop.init()
├── keyboardManager.init()
└── eventManager.init()
```

Получается жизненный цикл:

```
dependencies available
      v
DOM available
      v
component state initialized
      v
component event contracts established
      v
application routing activated
```

6. Состояние уровня приложения

ShowcaseApp хранит только то состояние, которое нужно для координации.

Например:

```
_isSliderMoving
_btnNoActive
```

Он не дублирует:

```
slider current index
autoscroll FSM
audio current track
audio playback mode
```

Они остаются у своих владельцев.

Это один из самых важных ownership boundaries в архитектуре.

7. Application Event Map

Основная карта событий включает browser events и component-generated custom events.

Browser

click
auxclick
keydown
keyup
mousedown

Slider

viewportclick
slidemove
slidechange
autoscrollchange

AudioPlayer

albumplay
albumpause
albumplaypassthrough
albumend
audiotrackchange
timechange

Каждый route описывает:

target
handler
optional event options

EventManager превращает эту декларацию в реальные subscriptions.

8. `_stream()` и `_pipe()`

У этих методов разные ответственности.

`_stream()`

└─ выбирает порядок pipeline

`_pipe()`

└─ исполняет выбранный pipeline

Обычный вариант:

[Slider, AudioPlayer, Shop]

Специальный reverse-вариант:

```
[ AudioPlayer, Slider, Shop ]
```

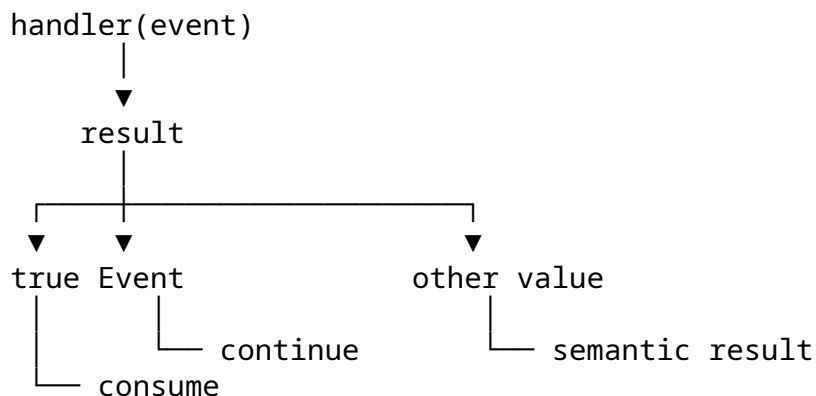
Сам цикл находится в `_pipe()`.

```
for component in pipeline
  |
  | value всё ещё Event?
  |   | иначе -> вернуть его
  |
  | у component есть handler?
  |   | нет -> следующий component
  |   | да  -> handler(event)
```

Так выбор порядка отделён от механики исполнения pipeline.

9. Return protocol `_pipe()`

Это один из самых интересных небольших архитектурных механизмов проекта.



Если handler возвращает:

`true`

верхний уровень может интерпретировать событие как поглощённое.

Если action/manager не возвращает результат, исходный event сохраняется:

```
return request?.action(...) ?? e;
```

Таким образом:

`original event`

становится continuation token.

Именно поэтому pipeline остаётся небольшим: для него не потребовался отдельный PipelineResult object.

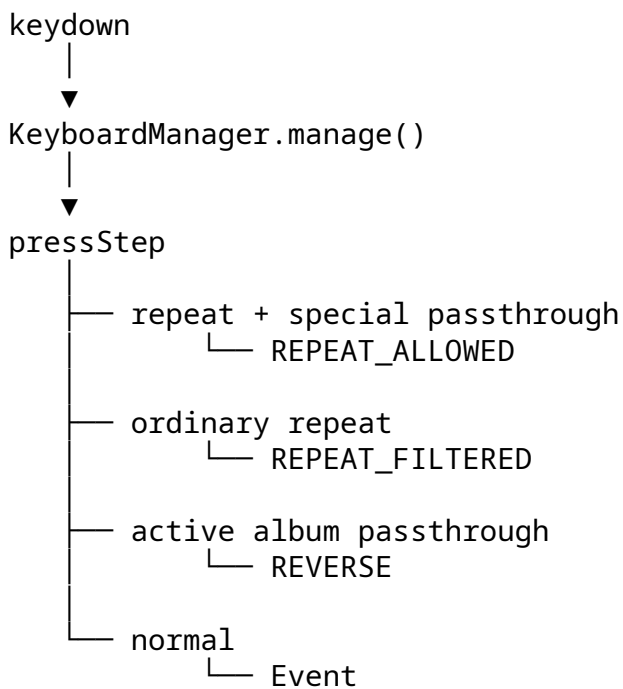
10. `_pressStep()` и keyboard modes приложения

Сначала keydown проходит через keyboard manager.

Затем ShowcaseApp может классифицировать результат в application-level modes:

REVERSE
REPEAT_FILTERED
REPEAT_ALLOWED
ordinary Event

Упрощённо:



Менеджер распознаёт действие, а application layer решает, как особый ввод должен распространяться по всей системе.

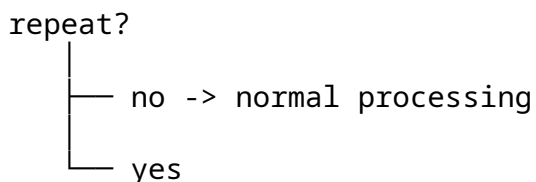
11. Repeat filtering

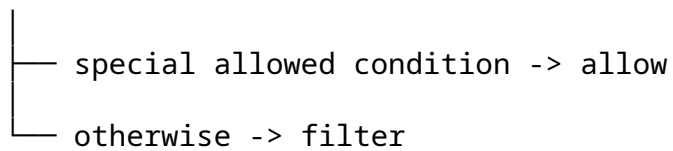
Повторный keydown не обрабатывается одним универсальным правилом.

Приложение проверяет:

`e.repeat`

и учитывает текущее audio/passthrough состояние.





Это позволяет long-running key press вести себя иначе, чем одноразовое действие.

12. Reverse input routing

Во время активного album playback специальный passthrough key может требовать, чтобы AudioPlayer получил событие раньше Slider.

normal

Slider -> AudioPlayer -> Shop

reverse

AudioPlayer -> Slider -> Shop

Это одна из причин, почему фиксированной цепочки было бы недостаточно. Порядок pipeline сам становится частью application policy.

13. Keyboard commands

Приложение использует набор semantic commands.

Для Slider это концептуально:

next
prev
autoscroll
execute
toggleautoscroll
reset
ignore

Для AudioPlayer:

next track
previous track
play
pause
toggle
restart
track selection
album selection

Клавиши браузера являются input data:

ArrowRight
KeyD
Enter
Space
Escape
...

Менеджер переводит физический input в semantic action.

14. Numeric / Shift audio track selection

Прямой выбор track поддерживается через key input.

```
keyboard symbol
  |
  ▼
track-number mapping
  |
  ▼
audioTrackInQueue setter
  |
  ▼
validated track index
```

Таким образом, валидность queue position остаётся ответственностью AudioPlayer, а не каждого caller.

15. Auxiliary mouse handling

Application layer различает:

left
middle
right

В частности, right click игнорируется в auxiliary-click flow. Middle click может преобразовываться в обычный application click, если он пришёл с interactive target.

Это этап нормализации браузерной семантики:

```
native mouse semantics
  v
application semantics
  v
component pipeline
```

16. Временное состояние правой кнопки

При right mousedown над кнопкой применяется временный visual class.

```
right mousedown
  |
  ▼
find button
  |
  ▼
remember _btnNoActive
  |
  ▼
apply jsClasses.btnNoActive
  |
  ▼
dynamic mouseleave subscription
```

На mouseleave:

```
remove visual class
unsubscribe dynamic event
clear _btnNoActive
```

Это конкретный пример temporary application state + dynamic EventManager subscriptions.

17. Визуальное состояние keyboard press

После успешно поглощённого keyboard action:

```
keydown
  |
  ▼
pipeline
  |
  ▼
result === true
  |
  ▼
active element
  |
  ▼
keyboard pressed CSS state
```

keyup снимает визуальное состояние.

Таким образом visual feedback является следствием semantic command result, а не отдельной реализацией в каждом component.

18. BaseSlider - граница ответственности

BaseSlider владеет общим Slider engine.

Его область включает:

- DOM discovery
- slide collection
- index management
- track movement
- navigation
- transition state
- resize handling
- common event map
- button primitives

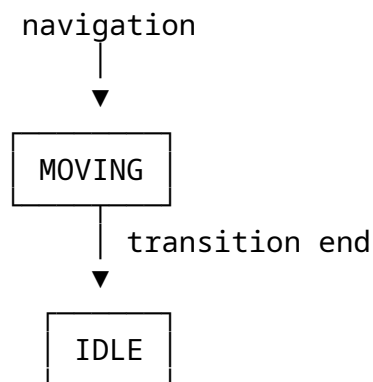
Поздние subclasses используют этот stable base contract.

Финальный Slider всё ещё предоставляет высокоуровневые команды:

- next()
- prev()
- goto()

несмотря на дополнительные уровни поведения.

19. Состояние базового Slider



Setter состояния валидирует state token.

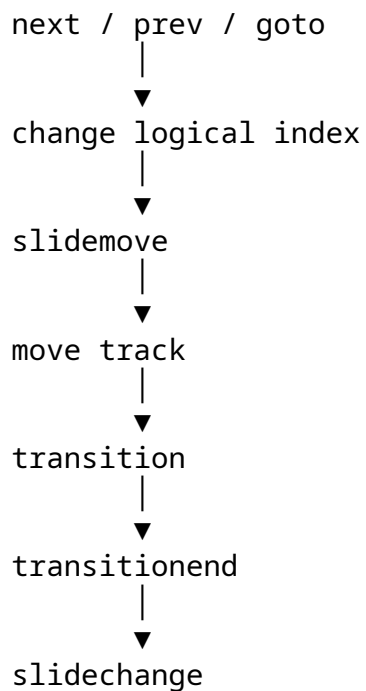
В polymorphic варианте архитектуры он может разрешать таблицу состояний через:

```
this.constructor.STATES[stateKey]
```

Это осмысленное место для runtime-class static polymorphism.

20. Lifecycle движения Slider

Концептуальная последовательность:



Проект намеренно разделяет:

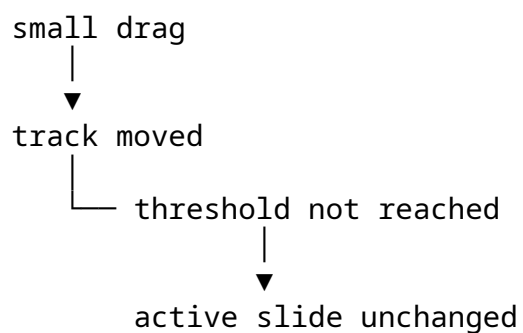
movement started

и:

active slide committed

21. Почему это различие важно

Например, небольшой drag может двигать track, не меняя active slide.



Поэтому нельзя приравнивать:

track movement = slidechange

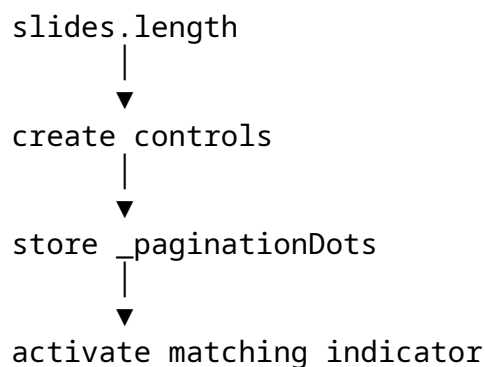
Это особенно важно для synchronization Slider <-> AudioPlayer <-> Shop.

Несколько пикселей движения не должны переключать album.

Этот случай стал хорошим примером того, как чёткая архитектурная граница помогает найти и исправить реальный поведенческий bug.

22. PaginationSlider

Pagination создаётся из данных о количестве slides.



Это хороший пример data-dependent UI.

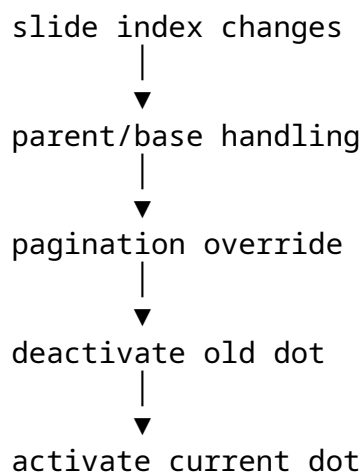
Постоянные navigation controls имеют другой жизненный цикл и не обязаны динамически генерироваться только ради одинакового стиля.

Принцип:

data-dependent UI	-> generate from data
fixed UI	-> can remain in markup

23. Pagination и index synchronization

Committed index change проходит через inherited hooks и приводит к обновлению pagination state.



Pagination layer не дублирует Slider movement algorithm.

24. KeyboardSlider

KeyboardSlider связывает Slider semantics с reusable KeyboardManager.

Разделение такое:

KeyboardManager
= key recognition + command routing

KeyboardSlider
= slider meaning of commands

Поток:

```
ArrowRight
  v
KeyboardManager
  v
next
  v
Slider method
  v
next slide
```

Компонент остаётся владельцем результирующих state changes.

25. Keyboard configuration как Strategy-like механизм

Keyboard mapping задаётся как configuration.

```
next -> ArrowRight / D
prev -> ArrowLeft / A
...
```

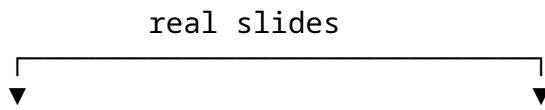
Менеджеру не нужен отдельный алгоритм для каждого компонента.

```
same manager
├── slider config
├── audio config
└── app config
```

Поэтому behavior selection происходит через data, что напоминает Strategy pattern.

26. InfiniteSlider

Infinite loop строится на clone boundary slides.



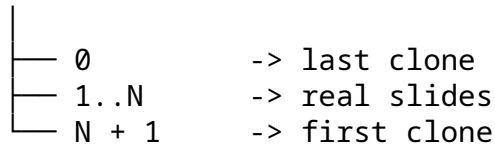
[last clone] [1] [2] ... [N] [first clone]

Clone positions существуют только для визуально непрерывного движения через границу.

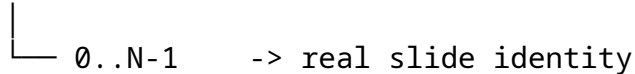
27. Infinite loop teleport

На границах DOM index и logical index расходятся.

DOM index



logical index



При достижении clone boundary:

boundary clone



reset/teleport



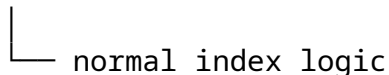
corresponding real slide

Остальная система не знает, что clone вообще существует.

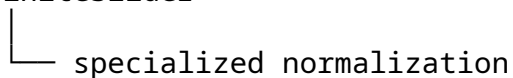
28. Index normalization

Смысл inheritance здесь в том, что базовый API может оставаться тем же, а Slider слой меняет физическое значение index.

BaseSlider



InfiniteSlider



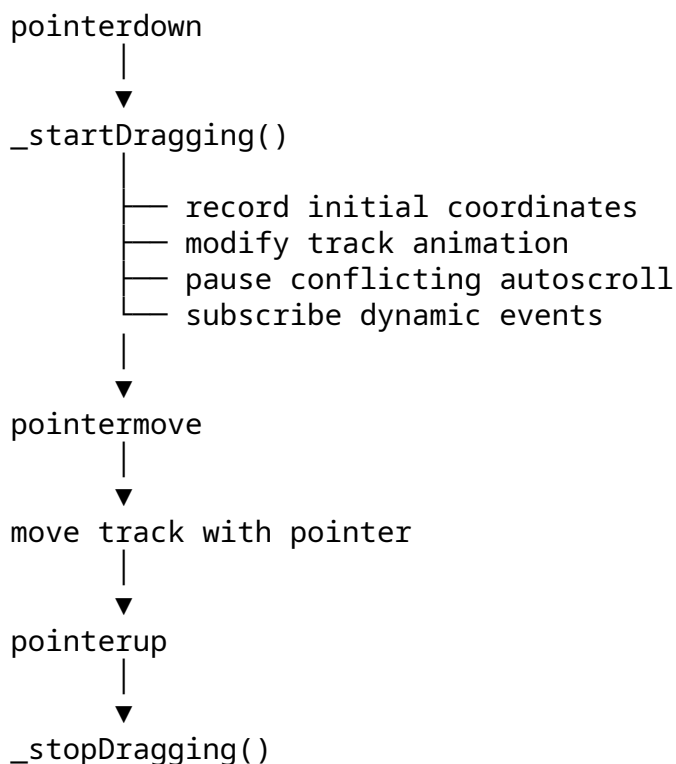
Внешний код по-прежнему использует:

```
next()  
prev()  
goto(index)
```

не зная о clone slides.

29. DraggableSlider lifecycle

Drag добавляет прямое управление указателем.



30. Drag trigger threshold

Drag не обязан менять slide автоматически.

Финальное смещение сравнивается с `threshold`, зависящим от геометрии.

```
trigger threshold  
=  
current slide width  
x  
configured coefficient
```

Это лучше, чем один жёсткий pixel constant, особенно после resize.

31. AutoscrollerSlider

Autoscroller добавляет поверх dragging собственную подсистему:

```
autoscroller timer
autoscroller state
pause conditions
lock/unlock
wake-up timing
automatic-action marker
```

Timer остаётся generic:

```
Timer
└─ callback -> Slider._nextAuto()
```

Slider решает, что означает callback.

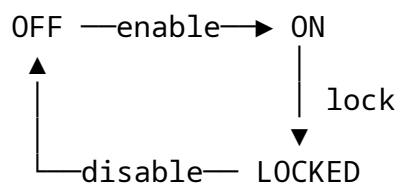
32. Autoscroller State Machine

Семантика состояний:

```
OFF
    automatic motion disabled

ON
    automatic motion allowed

LOCKED
    automatic motion temporarily prohibited
```

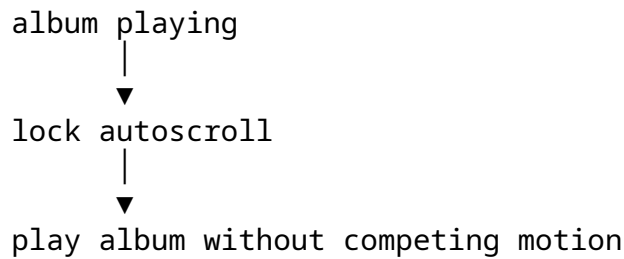


33. Почему LOCKED не равен OFF

OFF
= возможность отключена

LOCKED
= возможность существует, но временно запрещена условием

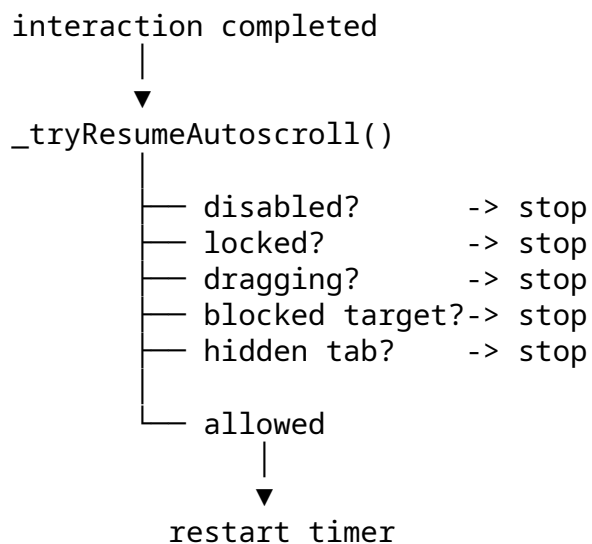
Например:



При паузе или завершении album приложение может решить, разрешать ли resume.

34. Autoscroll resume gate

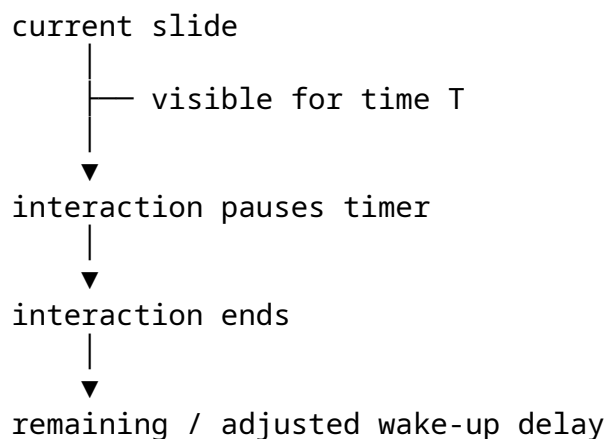
Автозапуск после взаимодействия не происходит автоматически.



Централизация этих условий защищает код от множества разрозненных start () calls.

35. Autoscroll wake-up timing

Различаются normal interval и restart/wake-up delay.



↓
normal autoscroll cycle

Смысл в том, чтобы hover или другое временное взаимодействие не сбрасывали весь интервал отображения slide без причины.

36. Automatic vs manual movement

Slider хранит контекст автоматического запуска:

`_isAutoscrollAction`

Условно:

```
manual movement
└─ normal interaction logic

automatic movement
└─ _isAutoscrollAction = true
```

Это предотвращает применение manual-only side effects к движениям, инициированным autoscroll.

37. Autoscroll + dragging

Обе функции пытаются управлять одним track.

```
autoscroll ON
┆
┆ ↓
┆ drag starts
┆ ┆
┆ ┆ ┆ automatic movement suspended
┆ ┆ ┆ direct pointer movement enabled
┆ ┆ ┆ temporary subscriptions
┆ ┆
┆ ┆ ↓
┆ ┆ drag ends
┆ ┆
┆ ┆ ↓
┆ ┆ _tryResumeAutoscroll()
```

Это ещё одна причина, по которой autoscroll state должен быть более выразительным, чем boolean.

38. AudioPlayer - граница домена

AudioPlayer является отдельным stateful component.

Он владеет:

```
native Audio object
theme playback
album playback
track selection
album selection
playback commands
playlist progression
media-session behavior
audio FSM
```

Остальные компоненты не должны напрямую менять:

```
audio.src
audio.currentTime
audio state
playlist indices
```

Они используют semantic commands.

39. Public command surface AudioPlayer

Концептуальный API включает:

```
play()
pause()
toggle()

playTheme()
resetTheme()

nextAudioTrack()
prevAudioTrack()
switchAudioTrack()

nextAlbum()
prevAlbum()
switchAlbum()

restartAudioTrack()
restartAlbum()
restartPlaylist()

rewindAudioTrack()
rewindAlbum()
rewindPlaylist()
```

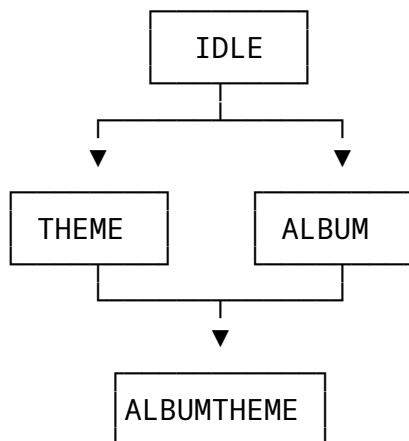
```
stopAudioTrack()  
stopAlbum()  
stopPlaylist()
```

Главный принцип:

```
public semantic operation  
    >  
native media implementation detail
```

40. AudioPlayer FSM

Player моделирует application-level playback modes.



Это не просто mirror native <audio> state. Это semantic mode model приложения.

41. Почему native Audio state недостаточно

audio.paused не говорит, что именно было поставлено на паузу:

paused main theme

или:

paused album

Для браузера оба случая означают paused. Для приложения - нет.

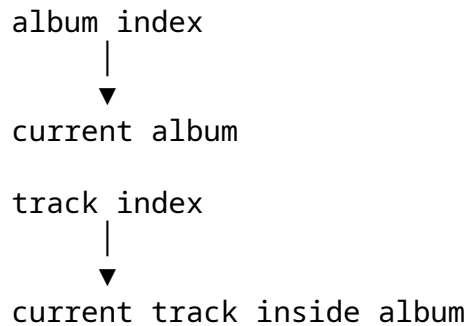
Поэтому:

```
native media state  
    +  
application playback context  
    v  
AudioPlayer FSM
```


Это частный случай общего принципа: browser state не всегда совпадает с domain state.

42. Album и track indices

Player хранит отдельные позиции:



audioTrackInQueue setter валидирует и нормализует queue position.

Это сохраняет инвариант:

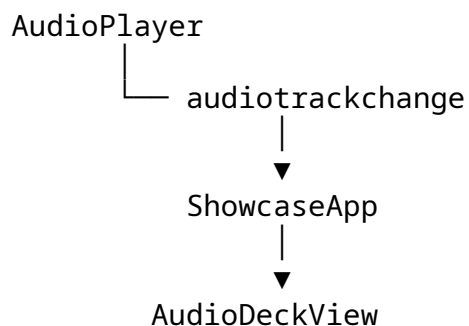
internal queue index is valid

43. Audio event model

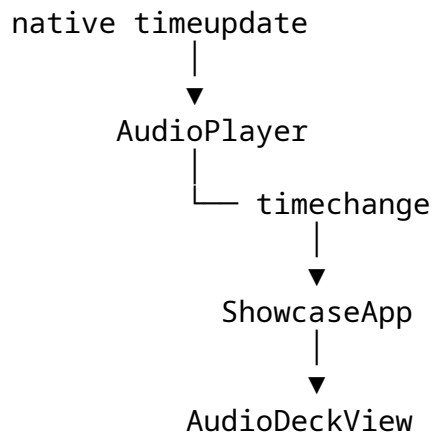
AudioPlayer публикует custom events:

albumplay
albumpause
albumplaypassthrough
albumend
audiotrackchange
timechange

Например:

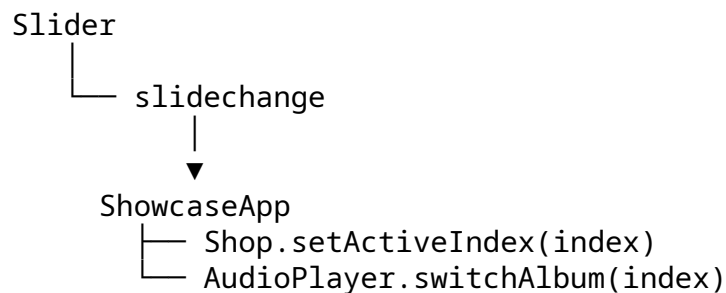


Для timeline:



44. Slider <-> Audio synchronization

Точка синхронизации - slidechange.



Slider сообщает только о смене logical slide identity.

Он не знает об альбомах, артистах или магазинах.

45. Почему slidechange является synchronization event

Небольшой drag может двигать track без изменения active slide.

Поэтому неверно синхронизировать album при каждом физическом movement.

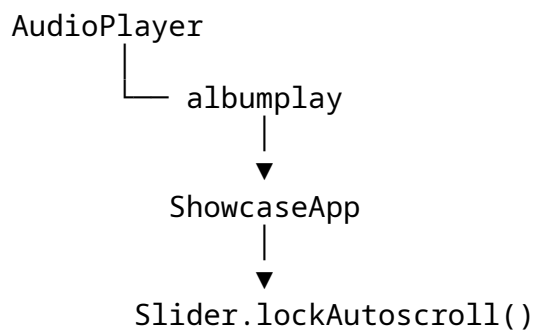
Нужно дождаться semantic event:

slidechange

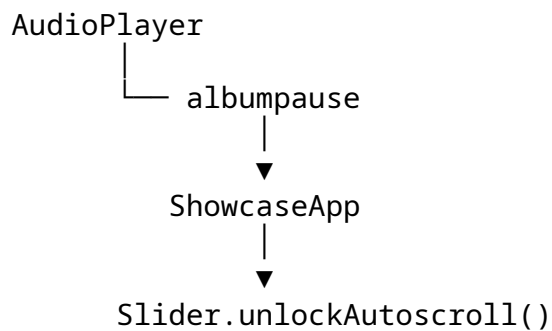
Так boundary между physical movement и semantic change становится защитой от ошибочной синхронизации.

46. Audio -> Slider synchronization

При старте album playback:



При паузе:



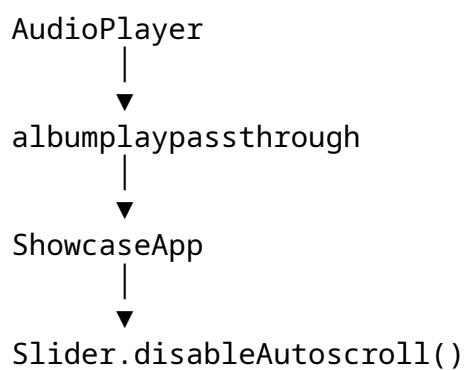
AudioPlayer не знает internals Slider.

47. Album passthrough event

Особое playback/input condition может привести к:

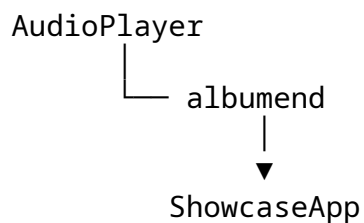
albumplaypassthrough

Application layer может преобразовать это в изменение autoscroll policy:



48. Album end

Когда альбом заканчивается:



Приложение переводит это в navigation command.

Если document active:

`Slider.next()`

Если document inactive:

`Slider.nextInstantly()`

Так application layer учитывает runtime environment.

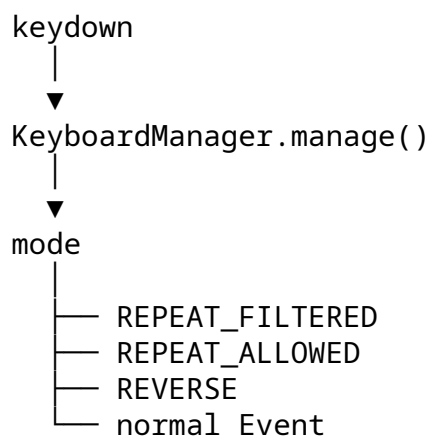
49. Input modes в ShowcaseApp

Keyboard handling включает не только key lookup.

Учитываются:

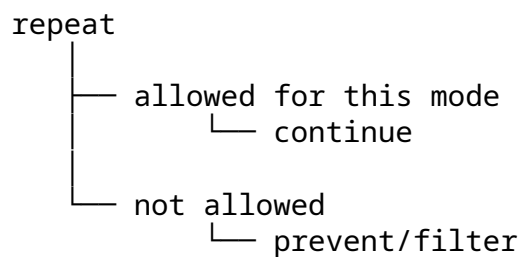
repeat event
passthrough condition
modifier state
reverse routing
execute mode
toggle mode

Концептуально:



50. Special repeat behavior

Not every repeat event is treated одинаково.



Это позволяет специально разрешать те действия, где повторение действительно имеет смысл.

51. Reverse pipeline

`_stream()` выбирает порядок, а `_pipe()` исполняет его.

normal
Slider -> AudioPlayer -> Shop

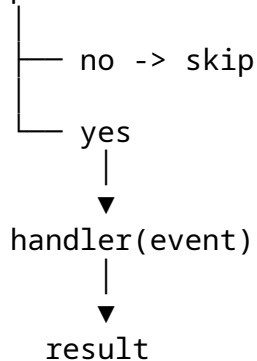
reverse
AudioPlayer -> Slider -> Shop

Это отделяет routing policy от механики итерации по pipeline.

52. `_pipe()` как контракт

Pipeline проходит по компонентам и обновляет текущий event/result.

does component have handler?



Интерпретация:

true
-> consumed

original Event
-> continue

other result
-> component/application-specific result

53. Почему это похоже на Chain of Responsibility

Структура не является textbook GoF class hierarchy с базовым Handler.

Но принцип совпадает:

```
handler A
├── handles -> stop
└── does not handle -> handler B
```

Поэтому точнее называть это лёгким Chain-of-Responsibility protocol через pipeline и polymorphic handlers.

54. Inheritance + parent handler cooperation

Specialized handler может сохранить parent logic и добавить собственную.

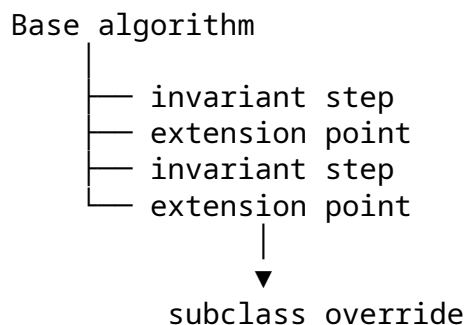
```
Child handler
  |
  ▼
Parent handler
├── consumed -> stop
└── passed -> child-specific behavior
```

Так постепенно складываются:

```
core navigation
  +
pagination
  +
keyboard
  +
infinite loop
  +
dragging
  +
autoscroll
```

55. Template Method-like structure

В тех местах, где базовая реализация задаёт общий алгоритм, а subclass переопределяет extension point, возникает Template Method-like structure.



Мы не распространяем это название на каждый override подряд - только на реальные алгоритмические extension points.

56. Static class contracts

Architecture использует static class-level contracts для:

- state tables
- event-map keys
- component-specific defaults
- configuration contracts

Главное различие:

fixed implementation constant

vs

polymorphic static contract

57. Module constants vs static properties

Module-level constant подходит для внутреннего фиксированного значения.

- STATES
- KEYBOARD_MODES
- mouse button constants

Static property/getter уместны, когда значение является именованным class-level contract:

- Class.STATES
- Class.EVENT_MAP_KEY
- Class.DEFAULT_URL

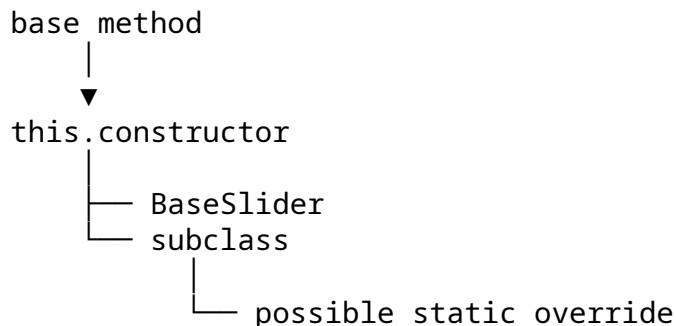
Поэтому проект не превращает каждую константу в static property.

58. this.constructor и polymorphic static lookup

Когда static contract должен следовать runtime constructor:

`this.constructor.STATES`

сохраняет subclass polymorphism.



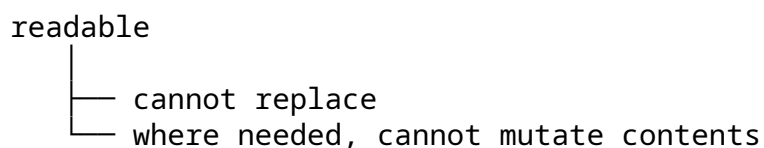
Это особенно уместно в state setters и других extension points.

Но использовать `this.constructor` повсюду только ради возможного будущего наследника не требуется.

59. Read-only static contracts

Когда static contract должен быть доступен для чтения, но не для замены, используется соответствующий read-only механизм.

Желаемая семантика:



Механизм вторичен по отношению к контракту.

60. Shop и default URL

Shop является маленьким примером class-level default contract.

Разделяются:

instance URL
= текущий target

static default URL
= fallback policy

Где нужен runtime-class polymorphism, fallback может разрешаться через actual constructor.

61. ButtonManager и KeyboardManager return contract

Оба менеджера используют общий result protocol.

```
action returns
├── true
│   └── handled
└── null/undefined
    └── preserve original event
```

Это позволяет использовать оба менеджера в одном pipeline без несовместимых протоколов.

62. Mouse и Button pipeline

Click event может проходить тот же pipeline.

```
MouseEvent
  ↓
ShowcaseApp
  ↓
Slider.handleClick()
  ↓
AudioPlayer.handleClick()
  ↓
Shop.handleClick()
```

Конкретный handler существует не обязательно у каждого компонента; infrastructure остаётся общей.

63. Auxiliary mouse input

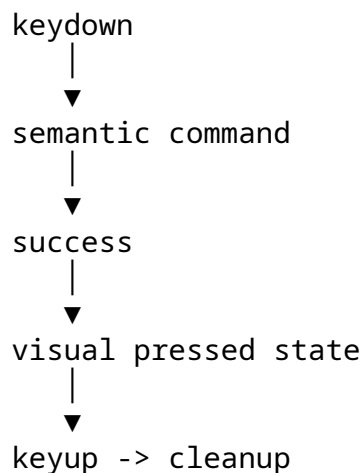
Application layer нормализует browser semantics:

```
left
middle
right
```

Это отделяет особенности DOM input от semantic commands компонентов.

64. Focus и keyboard UI state

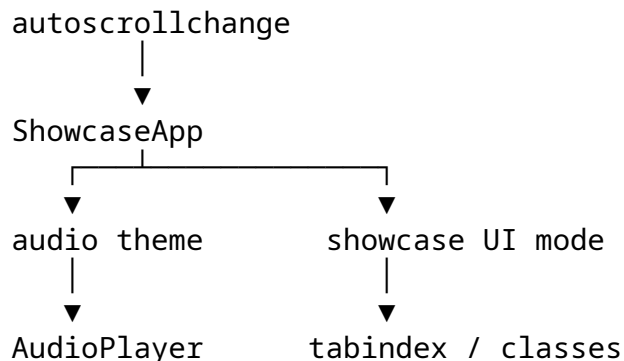
После успешного keyboard action application layer может включить временное pressed-state у соответствующего UI element.



Так keyboard visual feedback не дублируется внутри каждого доменного компонента.

65. Autoscroll и UI mode

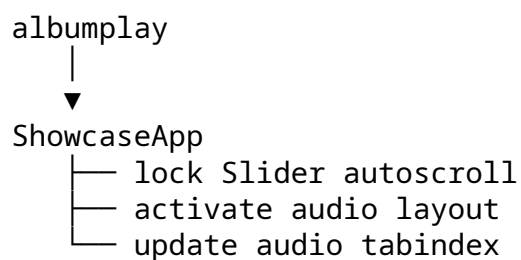
autoscrollbackchange сообщает не только о timer state.



Slider знает состояние autoscrollback. Application layer знает, какие последствия это имеет для продукта в целом.

66. Audio mode и UI mode

Album playback аналогично вызывает application-level последствия.



AudioPlayer остаётся сфокусирован на audio domain.

67. AudioDeckView

AudioDeckView намеренно маленький.

Он отображает:

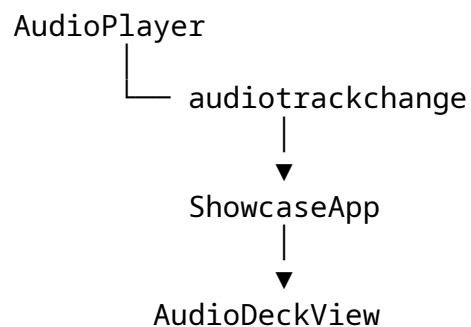
```
track title
track number / total
timeline
current time
duration
```

Он не решает:

```
which album plays
when track ends
which track is next
what playback mode exists
```

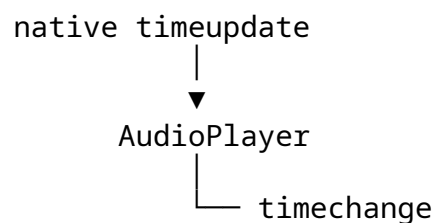
Это AudioPlayer/application concern.

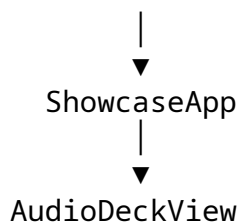
68. Audio track change



View получает достаточно данных для отображения, но не становится владельцем playlist state.

69. Audio timeline





Таким образом browser-specific media event остаётся внутри audio domain до момента, когда он превращается в application-level event.

70. Media Session

Player учитывает browser-level media controls.

В результате существует несколько внешних input sources:

- keyboard
- browser media shortcut
- native media event
- UI button

Все они должны в конечном итоге попасть в semantic commands:

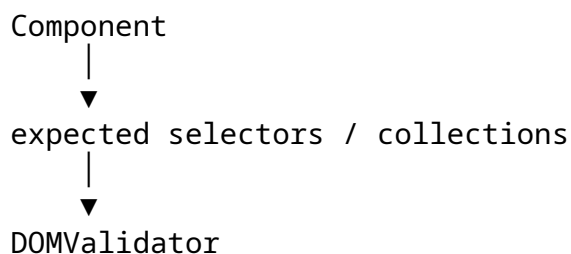
- play
- pause
- next
- previous

AudioPlayer выступает boundary, который нормализует эти источники.

71. DOM validation как explicit contract

Каждый major component ожидает определённую DOM structure.

Архитектура рассматривает её как API contract.



При нарушении required dependency возникает initialization error.

- explicit contract
- >
- silent recovery

72. Почему fallback selector guessing избегается

Если компонент ожидает конкретный selector, скрытый поиск по нескольким альтернативам может лишь замаскировать неверную структуру markup.

Проект предпочитает:

missing expected DOM



fail near initialization

Это делает architecture contract видимым.

73. Resize lifecycle

Slider geometry зависит от текущего viewport/track size.

resize



measure current geometry



recalculate slide offset



move track to current logical position

Это влияет не только на visual position, но и на drag threshold и infinite-loop positioning.

74. Dragging и Autoscroll interaction

Обе функции конкурируют за один track.

autoscroll ON



drag starts



automatic movement suspended
direct pointer movement enabled
temporary subscriptions



drag ends



_tryResumeAutoscroll()

75. Audio и Autoscroll - отдельные FSM

Одновременно могут существовать:

```
Slider state      = IDLE
Autoscroll state  = LOCKED
Audio state       = ALBUM
```

Это не противоречие, потому что эти состояния описывают разные domains.

Именно поэтому единая глобальная FSM была бы неудобной.

76. Configuration structure

Options разделяет категории:

```
singleSelectors
groupSelectors
classes
classesActive
jsClasses
states
click
press
autoplay
autoscrollDelay
autoscrollWakeUpDelay
slideTriggerThresholdCoef
```

Тем самым отделяются:

```
DOM naming
visual classes
runtime states
interaction rules
timing
```

от самих algorithms.

77. Configuration как contract

Configuration является больше чем bag of options.

Например:

```
press.next
  |
  ▼
expected handler
```

↓
▼
handler exists?

или:

```
event entry
├── target
└── handler
```

Менеджеры проверяют эти отношения во время initialization.

78. Micro-framework characteristics

Набор:

```
EventManager
KeyboardManager
ButtonManager
DOMValidator
Timer
```

уже напоминает маленький внутренний framework.

Но точнее говорить так:

Приложение выделило повторяющиеся архитектурные механизмы в configurable reusable services.

Ценность здесь не в новом названии, а в повторном использовании механики.

79. Что намеренно не абстрагировано

Не каждый повторяющийся элемент автоматически превращён в abstraction.

Например:

```
fixed state table
fixed implementation-specific constant
single-use local calculation
```

не требуют нового manager только потому, что принцип абстракции существует.

Это защищает проект от abstraction for abstraction's sake.

80. Почему State pattern не навязан

В проекте уже есть явные state tokens, setter validation и transition logic.

Поэтому отдельная State hierarchy была бы оправдана только тогда, когда state-specific behavior существенно разрастётся.

```
real problem
  v
needs state modeling
  v
local FSM is enough
  v
no extra abstraction
```

81. Более точная карта паттернов

Паттерн / принцип	Где	Почему подходит
Mediator-like	ShowcaseApp	централизует cross-component coordination
Observer / Pub-Sub-like	custom events + EventManager	decouples event producers and consumers
Chain of Responsibility	_stream() / _pipe()	handlers могут поглотить или передать input
Strategy-like	KeyboardManager, ButtonManager	action меняется через configuration
Template Method-like	Slider hierarchy	общий алгоритм и polymorphic extension points
Command-like	Button	UI action представлено как исполняемая команда
FSM	Slider / Autoscroll / AudioPlayer	state и transition локализованы у владельца
Composition	ShowcaseApp	система собрана из независимых частей
Dependency Injection	application construction	зависимости передаются снаружи
Inversion of Control	composition root	composition отделён от domain logic
Polymorphism	slider hierarchy / static contracts	специализация без полной дублирующей реализации

Точные оговорки:

DI	= архитектурная техника
IoC	= более широкий принцип
FSM	= technique for state modeling
Pub/Sub	= communication style
Observer	= родственный GoF concept, но реализация не textbook Observer
Factory	= термин уместен только при наличии реальной creation abstraction

82. Static inheritance в prototype implementation

В prototype-based implementation явно задаются две формы наследования:

Instance inheritance
Child.prototype



Parent.prototype

и:

Static inheritance

Child



Parent

Это позволяет class-level contracts участвовать в inheritance.

Поэтому в осмысленном polymorphic месте:

```
this.constructor.STATES
```

может сохранять возможность subclass override.

83. Discipline для aliases

Локальный alias полезен, если он реально улучшает читаемость при повторном использовании.

descriptive long name



short alias



many local uses

Но если static table нужна один раз, прямой доступ может быть яснее:

```
this.constructor.STATES.IDLE
```

То есть принцип не в том, чтобы никогда не делать alias, а в том, чтобы alias создавался ради ясности, а не автоматически.

83. Prototype version и Class version

Обе версии представляют одну conceptual architecture.

```
architecture
├── prototype syntax
└── class syntax
```

Prototype implementation делает более заметными:

```
prototype chain
constructor restoration
static inheritance
Object.setPrototypeOf(...)
```

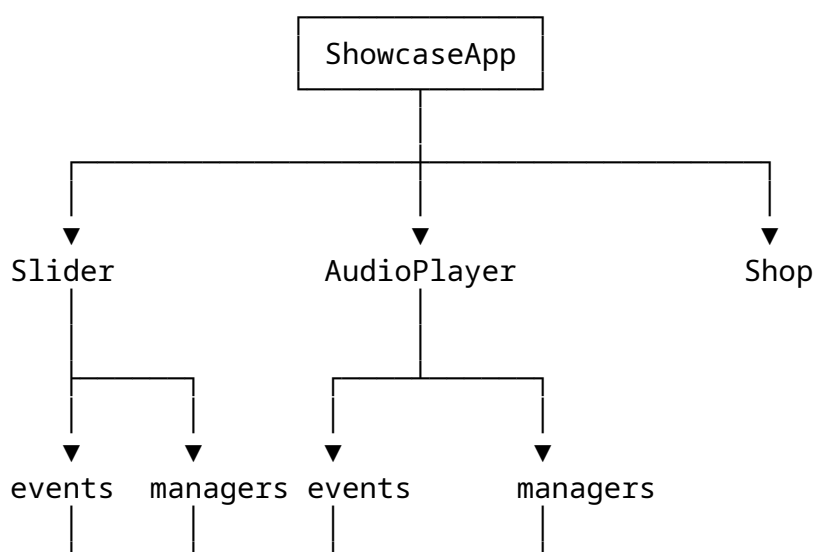
Class implementation выражает те же отношения через:

```
class
extends
static
super
```

Поэтому различие синтаксическое, а не архитектурное.

84. Маленькая внутренняя экосистема

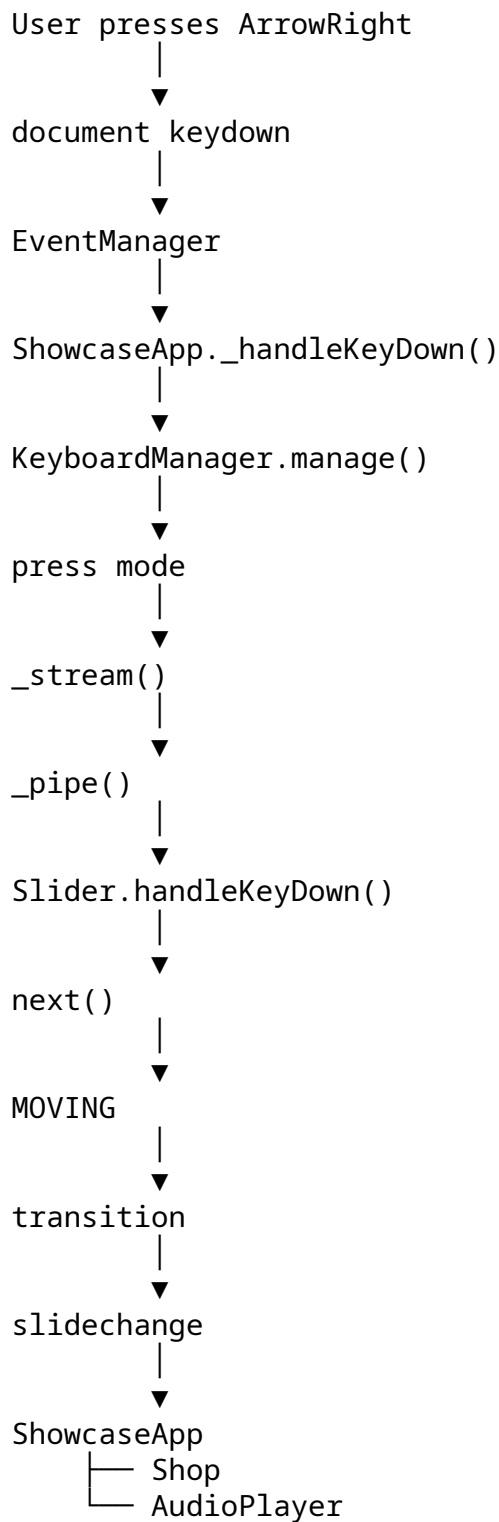
Во время runtime система выглядит как сеть специализированных объектов:



Система сложна, но у каждого объекта есть узнаваемая причина существования.

85. Typical user interaction - Keyboard

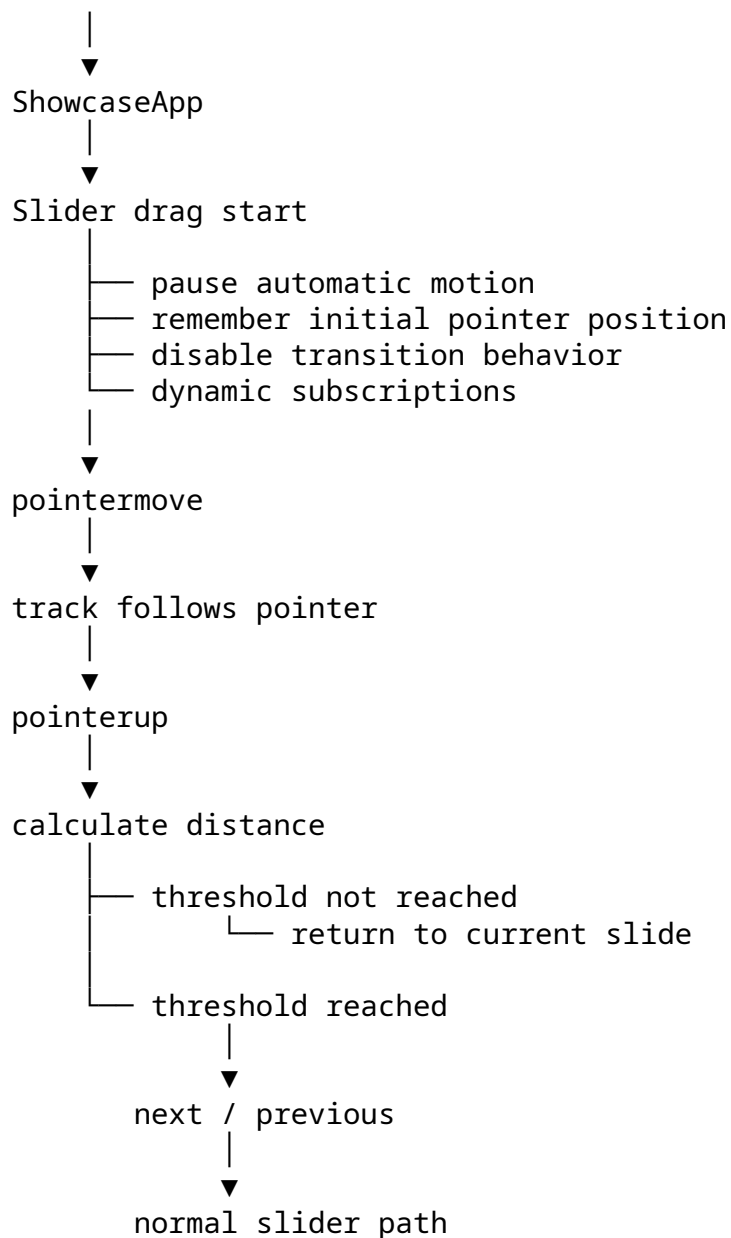
Один полный путь:



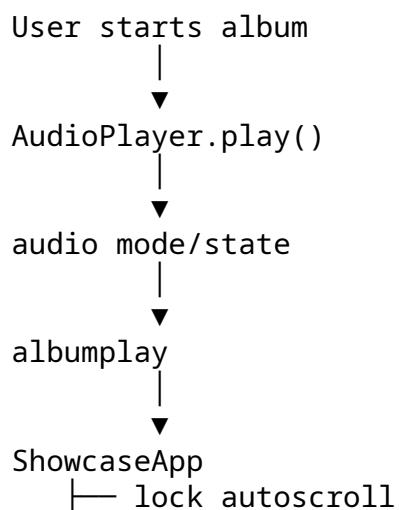
Здесь видна большая часть архитектуры в одном sequence.

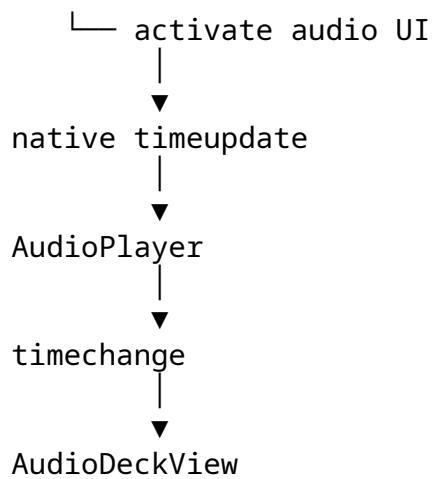
86. Typical user interaction - Drag

pointerdown

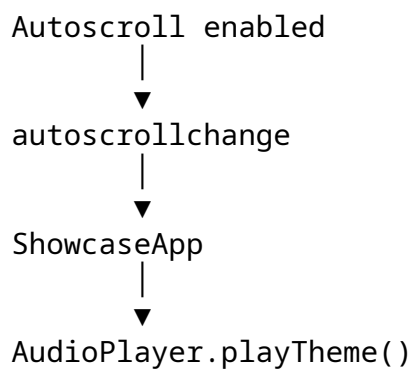


87. Typical user interaction - Album playback





88. Main theme / Autoscroll interaction



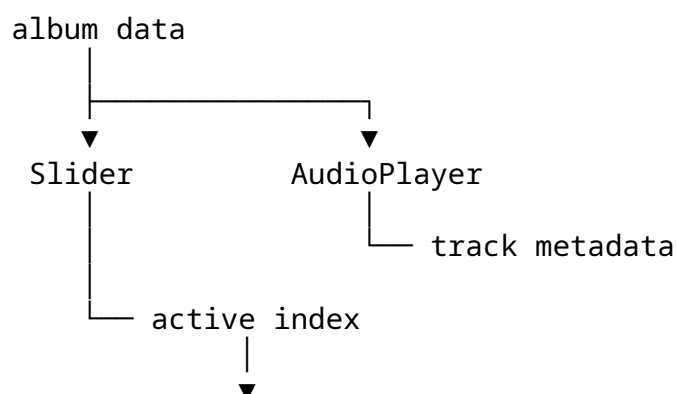
При выключении autoscroll приложение может выполнить:

`AudioPlayer.resetTheme()`

Это application policy, а не часть Slider internals.

89. Data flow

Album data является shared data source, но не global mutable state.



```
ShowcaseApp
├── Shop
└── AudioPlayer
```

Current slide index служит synchronizing identity между визуальным showcase и album data.

90. Component contracts

Public surface компонента должен быть semantic.

Примеры:

```
Slider
├── next()
├── prev()
├── goto()
├── lockAutoscroll()
├── unlockAutoscroll()
└── ...
```

```
AudioPlayer
├── play()
├── pause()
├── toggle()
├── playTheme()
├── switchAlbum()
├── switchAudioTrack()
└── ...
```

```
Shop
└── setActiveIndex()
```

Application layer предпочитает semantic command прямому mutation internals.

91. Error boundaries и validation

Проект проверяет разные категории assumptions:

```
DOM structure
event map structure
handler existence
target element availability
state tokens
index values
configuration values
```

Общий lifecycle:

validate at boundary

- valid -> continue
 - invalid -> explicit failure
-

92. Почему архитектура сложнее обычного Slider

Минимальная реализация могла бы использовать:

one object
one event handler
one interval
one audio element

Здесь вместо этого:

multiple component boundaries
multiple managers
multiple FSMs
multiple event layers
inheritance
polymorphism
configuration
validation
dynamic subscriptions

Это сознательный architectural trade-off.

93. Architectural trade-off

Больше abstraction означает:

more structure

- better separation
- reuse
- extensibility
- explicit contracts
- more code / mental overhead

Проект принимает этот cost, потому что architecture является частью учебной цели.

94. Что делает архитектуру coherent

Система coherent не потому, что в ней много classes.

Она coherent потому, что classes отвечают на разные вопросы:

ShowcaseApp
-> coordinates

Slider
-> controls slider behavior

AudioPlayer
-> controls audio behavior

AudioDeckView
-> renders audio information

Shop
-> controls external link state

EventManager
-> routes subscriptions

KeyboardManager
-> routes keyboard actions

ButtonManager
-> routes button actions

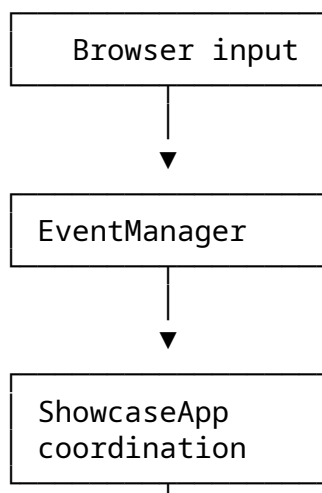
Timer
-> manages timing

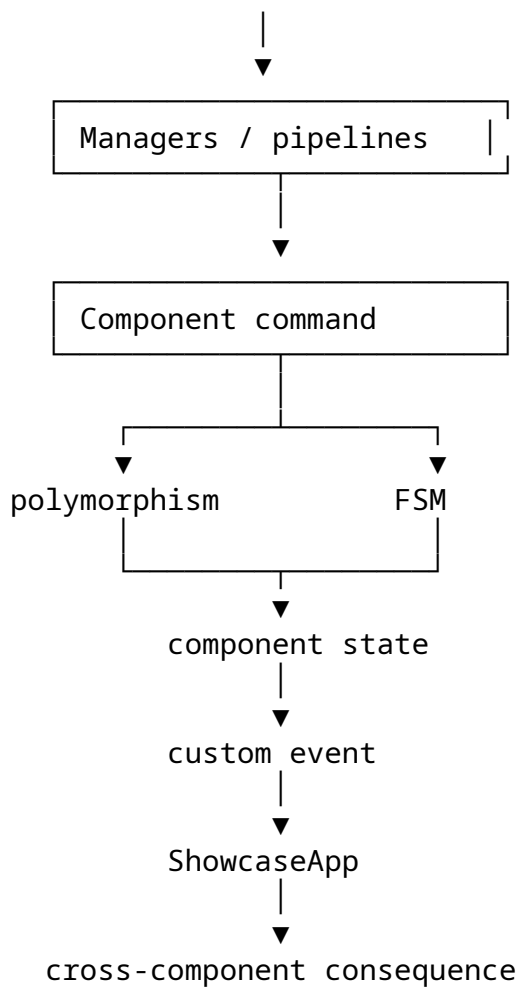
DOMValidator
-> enforces DOM contract

Button
-> represents executable UI action

95. Архитектурное ядро

Всю систему можно представить одним потоком:





Эта loop объясняет большую часть архитектуры проекта.

96. Final summary

Три FSM намеренно локальны:

Slider

└─ IDLE / MOVING

AutoscrollSlider

└─ OFF / ON / LOCKED

AudioPlayer

└─ IDLE / ALBUM / THEME / ALBUMTHEME

Main interaction model:

event

▼

routing

▼

pipeline

▼

polymorphic command
v
state transition
v
custom event
v
application coordination

И главная архитектурная идея:

Components own behavior.
Managers own reusable mechanisms.
ShowcaseApp owns composition and coordination.
FSMs stay with their owning components.
Patterns are named where the implementation genuinely resembles them.

97. Cross-Cutting Architectural Story

Почему все части действительно складываются вместе

Архитектуру легче всего запомнить как последовательность решений о владении ответственностью

1. Появляется browser event.
↓
2. Infrastructure его ловит и маршрутизирует.
↓
3. ShowcaseApp задаёт application-level order.
↓
4. Manager переводит raw input в semantic action.
↓
5. Владельческий component выполняет action.
↓
6. При необходимости меняется local FSM/state.
↓
7. Component публикует meaningful event.
↓
8. ShowcaseApp применяет cross-component consequence.

Это сквозная линия всего проекта.

Почему managers - больше, чем удобные wrappers

Менеджер оправдан тогда, когда он выносит механизм, который иначе начал бы повторяться или смешиваться с domain objects. KeyboardManager, например, владеет key matching, configuration interpretation и проверкой handler contract. ButtonManager делает

аналогичное для button/action mappings. EventManager владеет subscription lifecycle и discovery event maps. Timer владеет interval lifecycle.

Без managers

```
component
├─ addEventListener()
├─ removeEventListener()
├─ key matching
├─ button lookup
├─ handler validation
└─ domain behavior
```

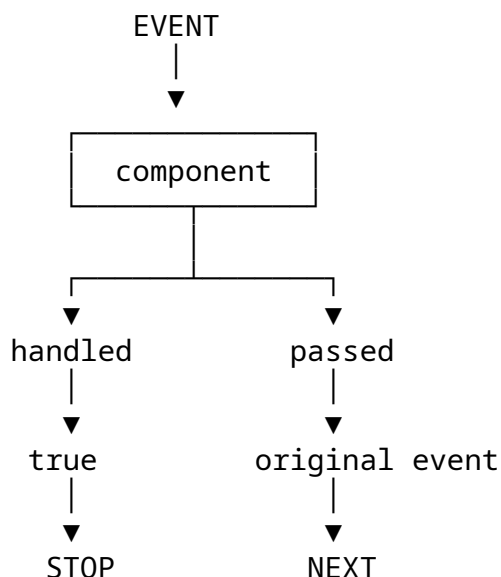
C managers

```
component
└─ domain behavior
    └─ manager handles reusable mechanism
```

Результат - не столько меньше строк, сколько более точный язык компонентов.

Почему `_pipe()` достоин отдельного описания

`_pipe()` - одно из мест, где сразу встречаются composition, polymorphism и explicit return-value protocol.



Pipeline поэтому является не просто utility loop, а маленьким protocol of cooperation.

Почему локальные FSM здесь предпочтительнее

Состояния проекта относятся к разным domains:

```
Slider FSM      -> movement lifecycle
Autoscroll FSM  -> automatic-motion permission/lifecycle
Audio FSM       -> playback context
```

Если попытаться слить их в одну macro-state model, появится матрица комбинаций вместо более ясной модели.

Local FSM сохраняет ownership близко к коду, который понимает это состояние.

Почему архитектура не гонится за паттернами

Паттерн полезен здесь, когда он даёт название реальной структуре кода. Он становится вредным, когда код начинают сгибать ради доказательства того, что паттерн применён.

Поэтому используются формулировки:

Template Method-like
Mediator-like
Command-like
Strategy-like
Observer/Pub-Sub-like

там, где соответствие частичное или адаптировано к native JavaScript. Архитектура первична; название паттерна - лишь полезный язык для обсуждения.

Учебная цель

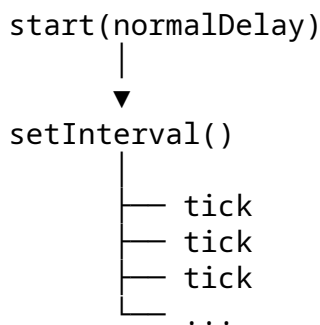
Проект построен вокруг сравнительно простой продуктовой идеи. Slider и audio showcase достаточно малы, чтобы оставаться полностью понятными, но при этом достаточно насыщены, чтобы показать реальные вопросы interaction, state, timing, event propagation, inheritance и cross-component coordination.

Поэтому проект работает как study object: UI даёт архитектуре конкретную задачу, а архитектура превращает UI в лабораторию, где ООП и system-design ideas можно исследовать без фреймворка, который скрывает большую часть механизмов.

98. Timer: зачем существует bootstrap

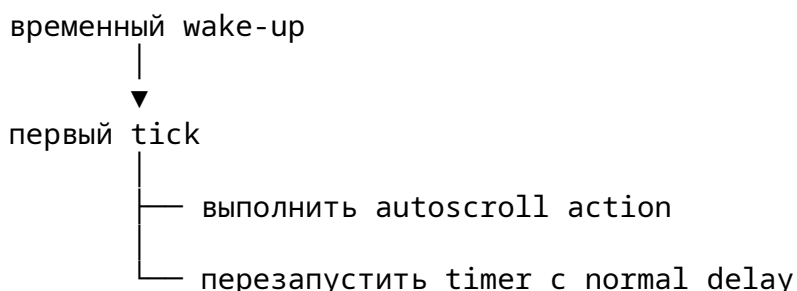
Timer выглядит небольшим utility-классом, но у него есть архитектурно важная деталь - bootstrap.

Обычный interval имеет простой жизненный цикл:



Но автоскроллу иногда нужен временный первый интервал, после которого необходимо вернуться к обычному интервалу.

Например:



Именно для этого существует bootstrap.

99. Как реально работает Timer.bootstrap

Timer хранит:

```
_delay  
_bootstrap  
_onTick
```

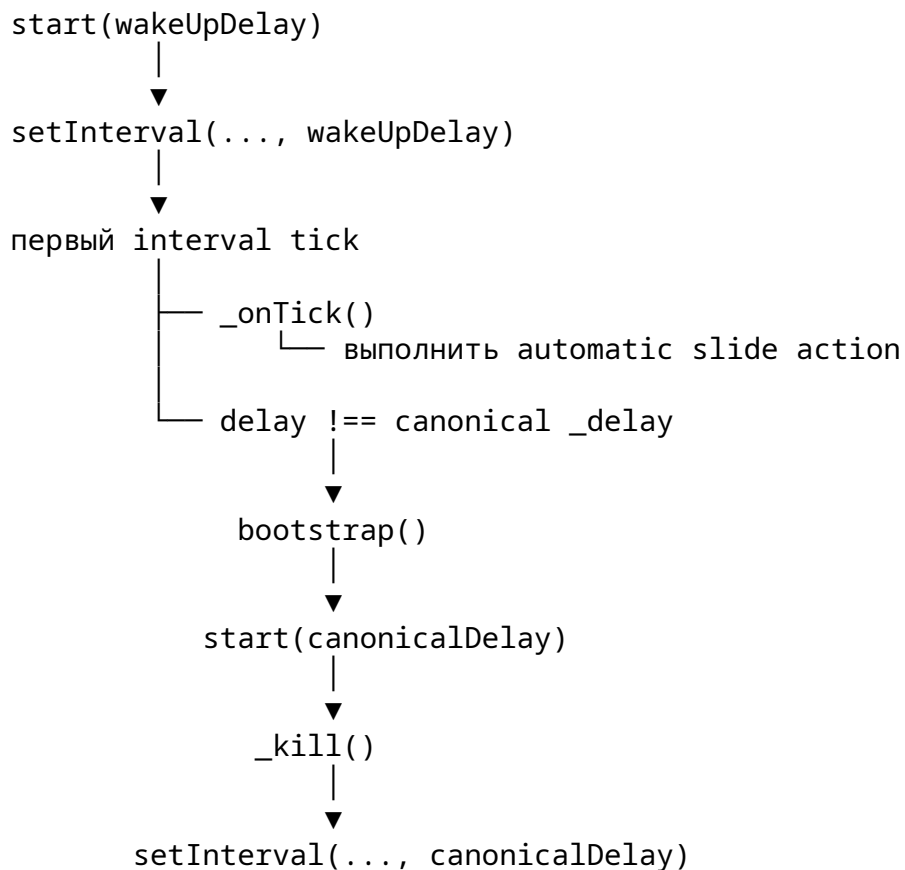
a start(delay) запускает interval через:

```
_tick(delay)
```

Ключевой фрагмент:

```
_tick(delay) {  
    this._onTick();  
  
    if (this._bootstrap && delay !== this._delay) {  
        this._bootstrap(this._delay);  
    }  
}
```

Получается такая последовательность:



То есть bootstrap - это не отдельный таймер.

Это **одноразовый переход от временного режима планирования обратно к нормальному режиму**.

100. Почему bootstrap находится в Timer, а не в AutoscrollSlider

Можно было бы реализовать всё непосредственно в Slider:

```
AutoscrollSlider
├── запустить временный timeout
├── выполнить action
└── запустить обычный interval
```

Но тогда domain component пришлось бы знать детали interval bookkeeping.

В текущей архитектуре:

```
AutoscrollSlider
├── передаёт Timer:
│   ├── bootstrapMethod = start
│   └── canonicalDelay
└──
    └── Timer
        └── владеет scheduling mechanics
```

Timer не знает, **почему** interval должен измениться.

AutoscrollSlider не занимается низкоуровневым управлением самим interval.

Это хороший небольшой пример separation of concerns:

Timer
→ как переключать scheduling

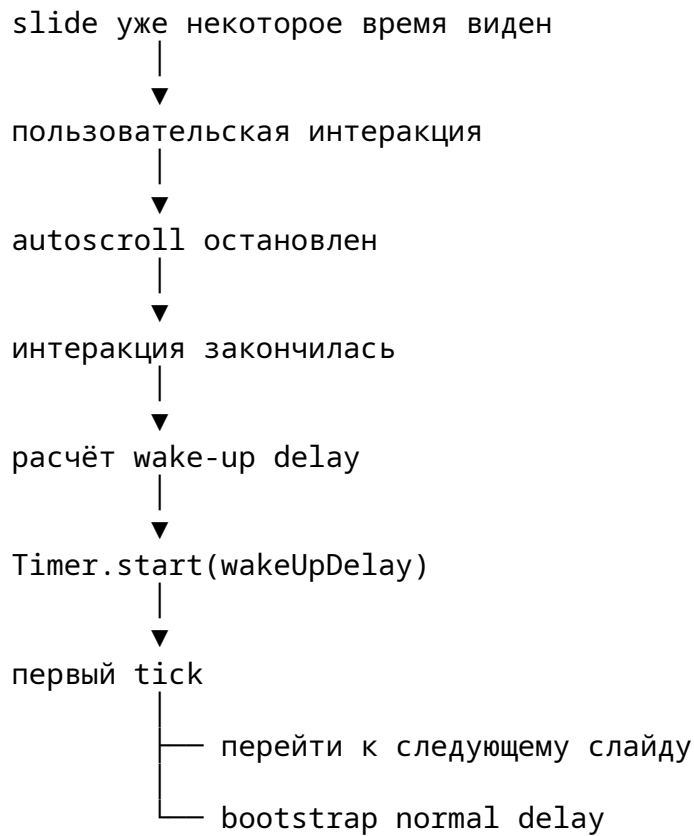
AutoscrollSlider
→ когда и зачем нужен временный delay

101. bootstrap и адаптивный autoscroll

Особенно хорошо это видно вместе с:

```
AUTOSCROLL_DELAY
AUTOSCROLL_WAKE_UP_DELAY
_getAdaptiveWakeUpDelay()
```

Полный смысл:



Таким образом Slider решает, **какой временный delay нужен**, а Timer реализует механизм возвращения к canonical interval.

Это важная граница между domain logic и infrastructure.

102. albumend -> Slider -> slidechange -> AudioPlayer

Один из наиболее интересных архитектурных сценариев - цикл между AudioPlayer и Slider.

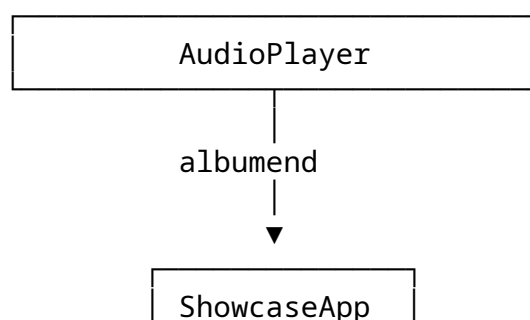
На поверхности его легко представить так:

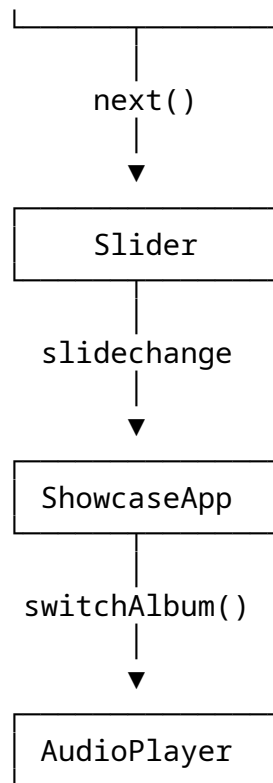
AudioPlayer -> Slider -> AudioPlayer

Но это неточное описание.

Нет прямой циклической зависимости между двумя объектами.

Фактический поток:





Это **event-mediated feedback loop** - цикл обратной связи, опосредованный событиями и application coordinator.

103. Почему этот feedback loop не является проблемой

Цикл соответствует реальному правилу приложения:

```
album закончился
↓
перейти к следующему визуальному элементу
↓
визуальный элемент определяет следующий album
↓
audio context синхронизируется с ним
```

При этом каждый компонент говорит только на языке своего domain:

AudioPlayer:
"мой album закончился"

ShowcaseApp:
"после окончания album нужно продвинуть showcase"

Slider:
"мой active slide изменился"

ShowcaseApp:
"после смены slide нужно синхронизировать album"

Ни `AudioPlayer`, ни `Slider` не обязаны знать внутреннюю реализацию друг друга.
Поэтому архитектурно это не:

`AudioPlayer` ↔ `Slider`

а:

`AudioPlayer` → event → `ShowcaseApp` → command → `Slider`
`Slider` → event → `ShowcaseApp` → command → `AudioPlayer`

104. `albumend` является `cancelable event`

`albumend` - не просто уведомление.

`AudioPlayer` создаёт его как:

```
new CustomEvent("albumend", {
  detail: { ... },
  bubbles: true,
  cancelable: true,
});
```

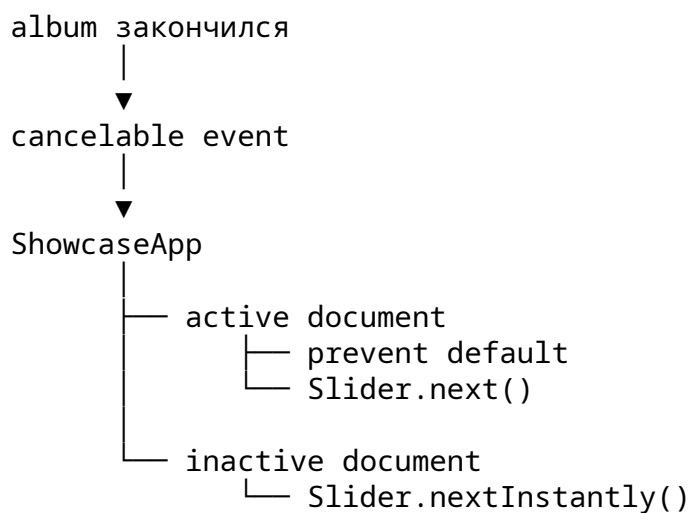
и затем проверяет:

```
return e.defaultPrevented;
```

Поэтому событие одновременно является:

notification
+
coordination point
+
optional control signal

Логика выглядит так:

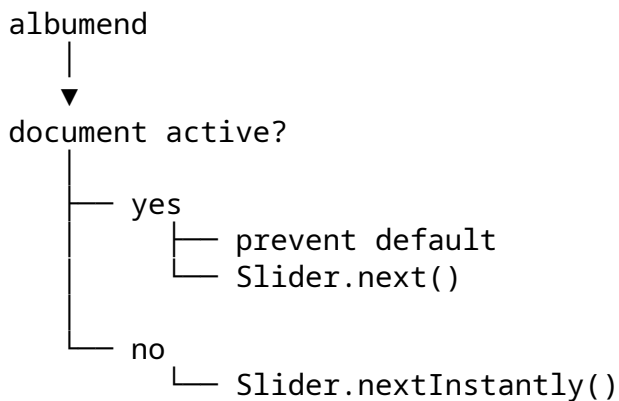


Это уже заметно более выразительный контракт, чем просто “отправить событие”.

105. Active document и background document

Приложение различает активную и неактивную вкладку/document.

Это влияет на реакцию на albumend:



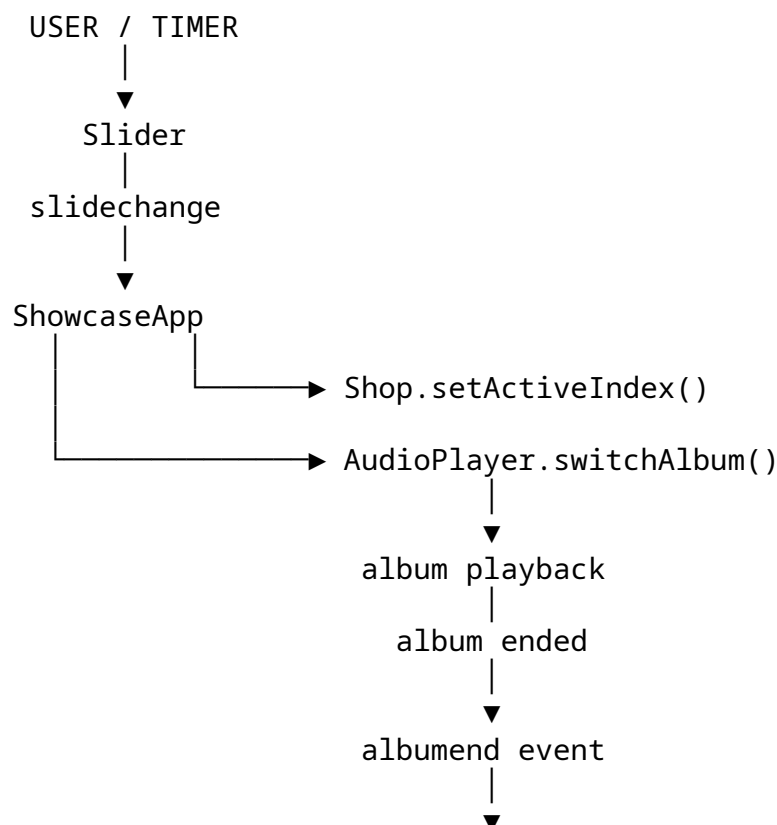
Причина проста: анимация имеет смысл, когда пользователь действительно смотрит на страницу.

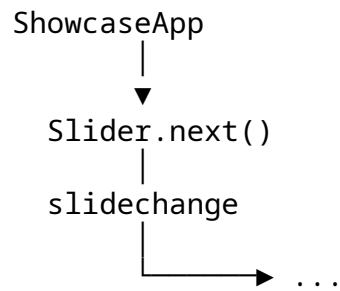
В background нет необходимости искусственно проигрывать визуальную последовательность.

Это хороший пример application policy, которая не принадлежит ни AudioPlayer, ни Slider по отдельности.

106. Полный synchronization loop

Если собрать несколько механизмов вместе:





Это намеренный цикл синхронизации двух представлений одного концептуального объекта:

```

visual showcase item
    ↑
audio album
  
```

107. Почему цикл не становится бесконечной цепочкой

Архитектура содержит важные guard conditions.

Например:

```

switchAlbum(index)
├── invalid → false
├── same album → без эффекта смены album
└── different album
    └── reset track
        └── try playback
  
```

И со стороны Slider:

```

slidechange
└── появляется только при committed logical slide change
  
```

Поэтому здесь нет неконтролируемой рекурсии.

Feedback loop есть на уровне **последовательности событий приложения**, но не на уровне прямого вызова методов друг через друга.

108. Intent и Event не смешиваются

AudioPlayer не говорит:

"переключи Slider на album X"

Он сообщает факт:

"album закончился"

ShowcaseApp преобразует этот факт в application command:

Slider.next()

И наоборот:

Slider:

"мой active index изменился"

ShowcaseApp:

"теперь нужно синхронизировать AudioPlayer"

Получается важная схема:

```
event
  ↓
fact
  ↓
application policy
  ↓
semantic command
```

Именно это позволяет компонентам оставаться независимыми.

109. Pipeline и custom events - разные механизмы

В проекте используются оба механизма, но для разных задач.

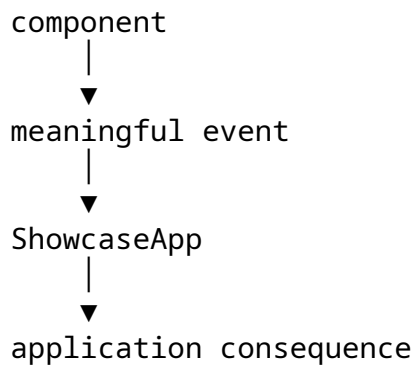
Pipeline

```
incoming shared input
  |
  ▼
component A
  |
  ▼
component B
  |
  ▼
component C
```

Он отвечает на вопрос:

Кто ещё должен получить этот input, если предыдущий обработчик его не потребил?

Custom event



Он отвечает на другой вопрос:

Что значимое произошло и кому нужно на это отреагировать?

Поэтому:

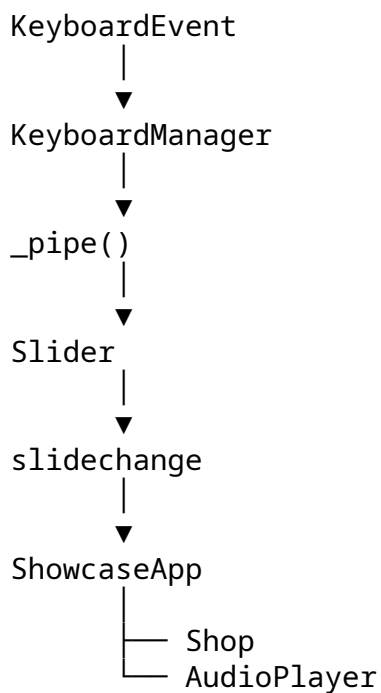
pipe
→ routing of incoming input

event
→ publication of application/domain fact

Это две взаимодополняющие модели.

110. Одна интеракция может пройти через обе модели

Например:



Сначала работает:

incoming command path

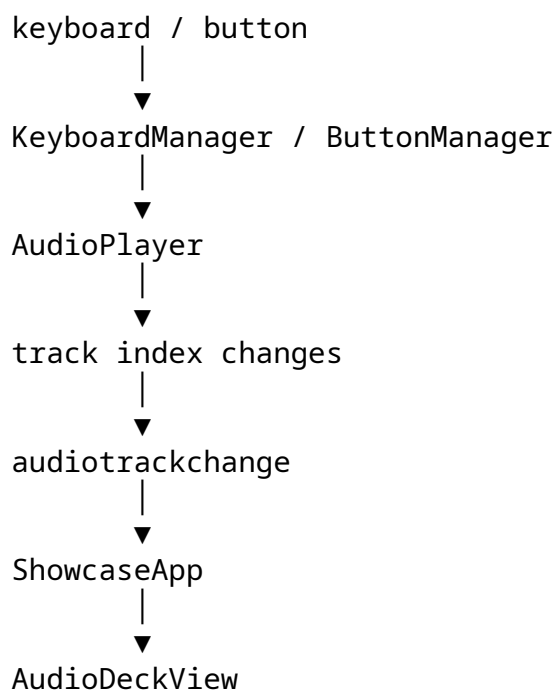
а затем:

outgoing domain-event path

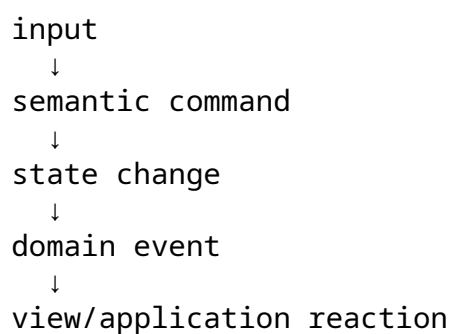
Это одна из причин, почему архитектуру нельзя свести только к “event-driven” или только к “pipeline-based”.

111. Аналогичный round trip внутри AudioPlayer

Для audio subsystem картина похожа:

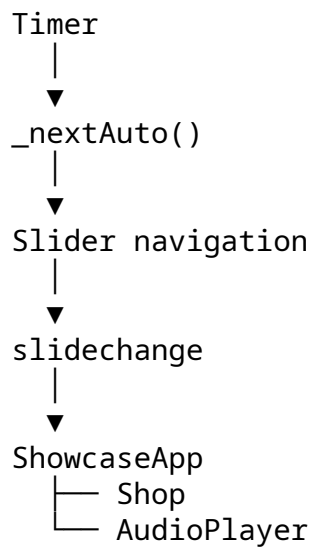


То есть повторяется общий architectural shape:



112. Timer и event architecture встречаются в autoscroll

Autoscroll особенно хорошо показывает, как несколько механизмов собираются в одну систему:



При этом Timer не знает ничего о:

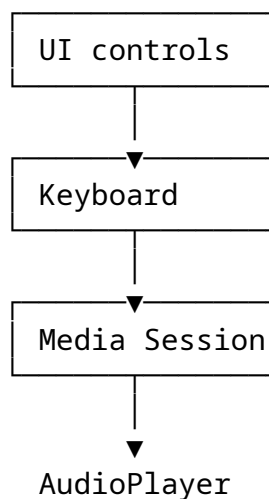
slides
albums
Shop
FSM

Он только запускает callback в нужный момент.

Это хороший пример того, почему даже маленькая infrastructure class может иметь смысл.

113. Media Session как ещё одна input boundary

AudioPlayer получает input не только из обычных DOM controls.



|
▼
semantic commands

Все эти внешние способы управления в итоге сходятся в semantic API AudioPlayer.
То есть компонент служит boundary между:

разными источниками input

и:

единым доменным API playback

114. Почему preventDefault() - не вся input architecture

Обычная обработка keydown и browser media controls могут идти разными путями.
Поэтому архитектура рассматривает:

обычный keyboard input

и:

Media Session input

как разные внешние каналы, которые сходятся внутри AudioPlayer.

Так browser-specific details остаются внутри audio boundary, а остальная система работает с:

play
pause
next
previous
toggle

115. State ownership в feedback loop

В цикле Slider ↔ AudioPlayer участвуют разные владельцы состояния:

Slider
└─ active visual index

AudioPlayer
└─ album / track / playback mode

ShowcaseApp
└─ application coordination state

Нет единого:

`global currentItem`

который все компоненты должны мутировать.

Вместо этого используются:

`local truth`

+

`events`

+

`coordination`

Так система остаётся синхронизированной без передачи ownership одному глобальному объекту.

116. Система event-driven, но не “глобально event-based”

Не каждый метод отправляет событие.

Не каждый вызов идёт через EventManager.

Механизмы выбираются по смыслу:

`обычный method call`

→ когда отношение между объектами уже принадлежит вызывающему

`custom event`

→ когда meaningful fact должен пересечь component boundary

`pipeline`

→ когда shared input должен пройти через несколько handlers

Это более точное описание архитектуры, чем просто:

"приложение использует events"

117. Ownership различных lifecycle

Разные механизмы жизненного цикла намеренно принадлежат разным объектам:

`DOM subscription lifecycle`

→ `EventManager`

`keyboard action lifecycle`

→ `KeyboardManager`

`button action lifecycle`

→ ButtonManager / Button

timer lifecycle

→ Timer

drag lifecycle

→ DraggableSlider

autoscroll lifecycle

→ AutoscrollSlider

audio lifecycle

→ AudioPlayer

cross-component lifecycle

→ ShowcaseApp

Это хорошо объясняет наличие нескольких небольших объектов вместо одного giant controller.

118. Почему маленькие объекты всё-таки образуют единую архитектуру

Некоторые abstractions сами по себе очень маленькие:

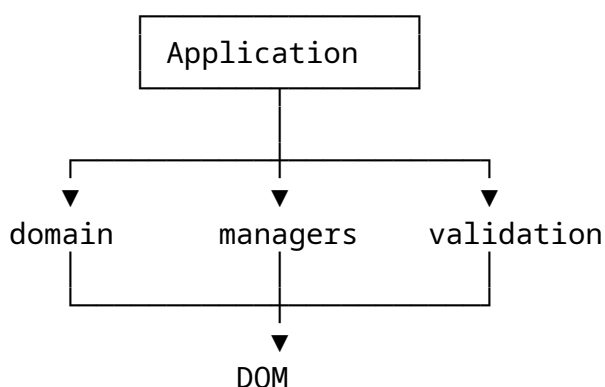
Timer

Button

Shop

DOMValidator

Их ценность становится видна на уровне composition:



Каждый объект снимает одну категорию ответственности с другого объекта.

Именно это, а не количество строк, является причиной его существования.

119. Эволюция архитектуры через реальные проблемы

Архитектура не была построена исключительно как теоретическая схема.

Проект развивался через реализацию, тестирование и debugging.

Это важно, потому что часть архитектурных решений появилась потому, что конкретное поведение обнаружило реальную проблему.

Например:

movement track
≠
committed slide change

стало существенным, когда небольшое движение каретки могло запускать неправильную downstream-обработку.

Аналогично:

automatic movement
≠
manual movement

важно для корректного restart/pause поведения autoscroll.

Поэтому архитектура отражает не только заранее выбранные patterns, но и проблемы, обнаруженные во время реализации.

120. Prototype/Class parity как отдельный архитектурный эксперимент

Вторая реализация - не просто механическая смена синтаксиса.

Она позволяет отдельно исследовать, что в архитектуре относится к:

prototype inheritance

а что относится к:

ES6 class syntax

Цель parity:

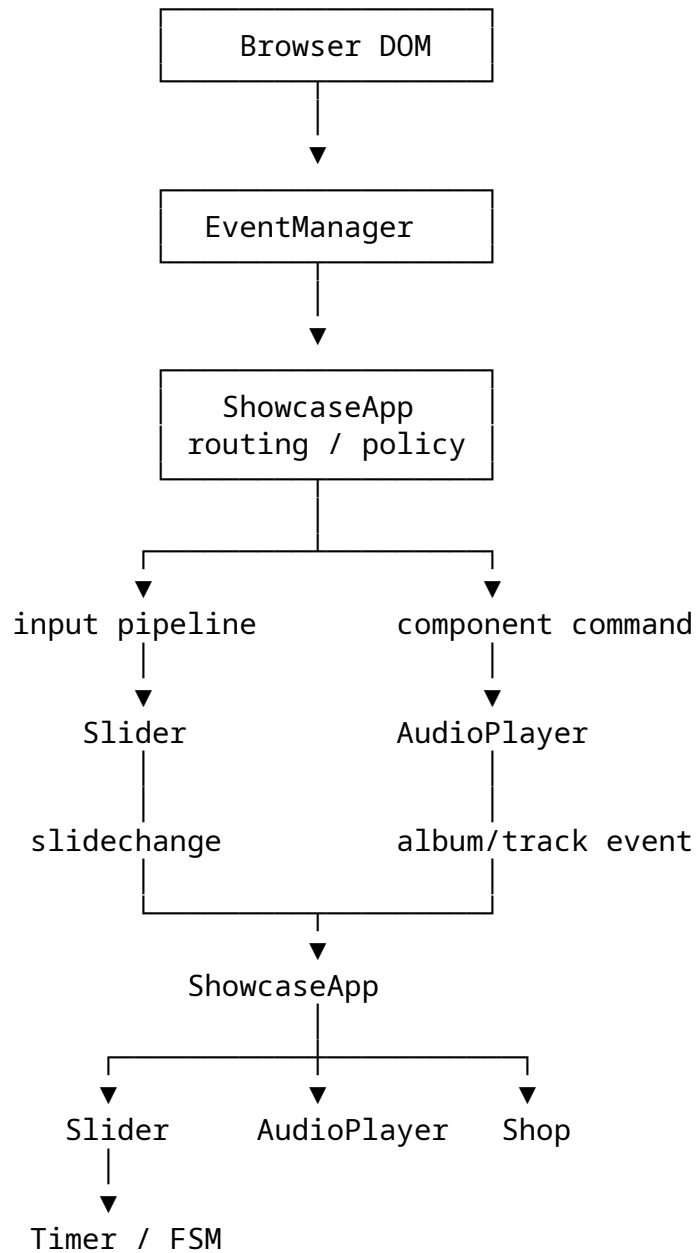
same responsibilities
same contracts
same FSM model
same event flow
same manager responsibilities
same component boundaries

при различающемся синтаксисе.

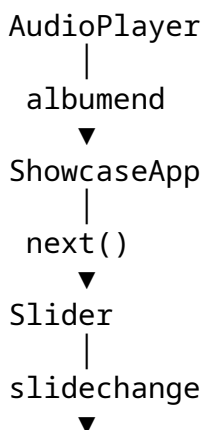
Поэтому архитектура в документации важнее конкретного способа записи inheritance.

121. Полная end-to-end картина

Всю систему можно представить одним большим, но понятным циклом:



И характерная feedback loop:



```
ShowcaseApp
|
switchAlbum()
▼
AudioPlayer
```

Это не случайная circular dependency.

Это координированный feedback loop, в котором:

```
events carry facts
ShowcaseApp carries policy
components carry domain state
commands carry intent
```

122. Заключительная перспектива Deep Dive

Наиболее полезно оценивать эту архитектуру не по количеству классов или названий паттернов.

Нужно смотреть на другое:

```
кто владеет решением?
кто выполняет механизм?
кто публикует факт?
кто координирует последствия?
```

И тогда основная схема становится очень простой:

```
input
↓
routing
↓
pipeline
↓
semantic command
↓
component algorithm
↓
local state
↓
domain event
↓
application coordination
↓
another semantic command
```

Главная идея проекта поэтому шире, чем просто “много архитектурных abstractions”:

Архитектура - это не коллекция абстракций. Это набор границ, который определяет, кто владеет решением, кто выполняет механизм, кто объявляет факт и кто отвечает за координацию его последствий.

И именно эта идея проходит через всю реализацию.